Self-Organizing Maps:

Nayan Pal
24AI10039

November 27, 2025

Abstract

Self-Organizing Maps (SOMs), also known as Kohonen maps [2], are unsupervised neural networks used for **clustering** and **nonlinear dimensionality reduction** while preserving the topological structure of data. This report describes the theoretical foundations of SOMs and evaluates their performance using two Python implementations: a from-scratch NumPy version and the MiniSom library. Experiments are conducted on the Iris dataset and a synthetic 2D blob dataset, including qualitative visualizations (U-matrix, hit maps, component planes) and quantitative measures (quantization error and cluster purity). The report also analyzes how grid size and learning parameters affect performance.

1 Introduction

High-dimensional datasets appear increasingly in applications like pattern recognition, finance, and bioinformatics, but directly visualizing and interpreting their structure is challenging. Unsupervised learning techniques such as clustering and manifold learning reveal intrinsic structure without requiring labeled data. Self-Organizing Maps [1] offer a way to map high-dimensional data onto a 2D grid of neurons while preserving neighborhood relationships, enabling both **clustering** and **visualization** in a single model.

This report covers three main areas:

1. The SOM architecture and learning algorithm, focusing on the competition–cooperation–adaptation paradigm
2. The Python implementations used, including hyperparameter selection and evaluation metrics
3. Experimental results on benchmark datasets, examining how SOM behavior changes with grid size, learning rate, and neighborhood radius

2 Theoretical Background

2.1 Architecture of a Self-Organizing Map

A SOM [2] consists of a regular low-dimensional grid (typically 2D) of units called neurons or nodes. Each neuron has an associated weight vector (or codebook vector) with the same dimension as the input data.

Let the input vectors be $x \in \mathbb{R}^d$, and each neuron i in the grid have a weight vector $w_i \in \mathbb{R}^d$. The neurons are arranged in a fixed lattice (rectangular or hexagonal), and each neuron has coordinates $r_i \in \mathbb{Z}^2$ showing its position on the grid.

Key idea: Nearby neurons on the grid should correspond to nearby regions in the input space. During training, weight vectors move so the grid forms an "elastic net" approximating the probability density of input data—high-density regions attract more neurons, while sparse regions get fewer.

2.2 Training Algorithm

Training a SOM is iterative, with three main phases for each input vector: *competition*, *cooperation*, and *adaptation*.

Given an input vector $x(t)$ at iteration t:

1. **Competition:** Compute the Euclidean distance between $x(t)$ and every weight vector $w_i(t)$, then find the **best matching unit (BMU)**:

$$c = \arg\min_i \|x(t) - w_i(t)\|_2.$$

2. **Cooperation:** Define a neighborhood around the BMU using a neighborhood function $h_{ci}(t)$, which is large for neurons close to the BMU and small for distant ones. A common choice is the Gaussian neighborhood:

$$h_{ci}(t) = \exp\left(-\frac{\|r_c - r_i\|_2^2}{2\,\sigma(t)^2}\right),$$

   where $\sigma(t)$ is the neighborhood radius.

3. **Adaptation:** Update the weights of the BMU and its neighbors:

$$w_i(t+1) = w_i(t) + \alpha(t)\, h_{ci}(t)\,\big(x(t) - w_i(t)\big),$$

   where $\alpha(t)$ is the learning rate.

2.2.1 Parameter Decay

Both learning rate $\alpha(t)$ and neighborhood radius $\sigma(t)$ typically decrease over time using exponential decay:

$$\alpha(t) = \alpha_0 \exp\left(-\frac{t}{T}\right), \qquad \sigma(t) = \sigma_0 \exp\left(-\frac{t}{T}\right),$$

where α_0 and σ_0 are initial values and T is a time constant proportional to maximum iterations. Early iterations with large $\sigma(t)$ encourage global ordering; later iterations with small $\sigma(t)$ allow fine-tuning.

2.2.2 Algorithm Summary

Input: Dataset X, grid size $(m \times n)$, iterations T
Output: Trained weight matrix W

1. Initialize weights $w_i \sim \mathcal{U}(0, 1)$ for all neurons.
2. For $t = 1$ to T:
   (a) Select a random sample $x(t)$ from X.
   (b) Find the BMU: $c = \arg\min_i \|x(t) - w_i(t)\|_2$.
   (c) Update learning rate: $\alpha(t) = \alpha_0 \exp(-t/T)$.
   (d) Update radius: $\sigma(t) = \sigma_0 \exp(-t/T)$.
   (e) For all neurons i:
      i. Calculate $h_{ci}(t) = \exp(-\|r_c - r_i\|^2/2\sigma(t)^2)$.
      ii. Update: $w_i(t+1) = w_i(t) + \alpha(t)h_{ci}(t)(x(t) - w_i(t))$.
3. Return W.

2.3 Computational Complexity

The time complexity of training a SOM for T iterations with N samples on an $m \times n$ grid is $\mathcal{O}(T \cdot d \cdot m \cdot n)$, where d is the input dimensionality. The BMU search requires $\mathcal{O}(m \cdot n \cdot d)$ comparisons per iteration. Space complexity is $\mathcal{O}(m \cdot n \cdot d)$ for storing the weight matrix.

2.4 Relation to Other Methods

The SOM algorithm is closely related to vector quantization and clustering methods like k-means. Like k-means, SOM performs competitive learning where prototype vectors adapt to represent clusters of similar inputs. However, SOM additionally enforces a neighborhood structure via $h_{ci}(t)$, encouraging nearby neurons to respond to similar inputs. This enables SOMs to function as both clustering algorithms and nonlinear projection methods for visualization.

Compared to linear techniques like PCA, SOMs can capture nonlinear manifolds, though at the cost of more complex training and hyperparameter tuning. SOMs also provide interpretable 2D maps (U-matrices [3] and component planes) that help users understand relationships between clusters and features.

3 Experimental Setup

3.1 Datasets

SOM implementations were evaluated on two datasets:

- **Iris dataset:** 150 samples of iris flowers with four continuous features (sepal length, sepal width, petal length, petal width) and three species labels, originally introduced by Fisher [4]. Labels are used only for evaluation, not training.
- **Synthetic blob dataset:** Created using scikit-learn's `make_blobs` with 300 samples and four well-separated clusters, demonstrating cluster formation on 2D maps.

Before training, each dataset was normalized using min–max scaling to $[0, 1]$, preventing large-scale features from dominating weight updates. Normalization is crucial because Euclidean distance is sensitive to feature scales.

3.2 Implementations

- **From-scratch NumPy:** Standard algorithm with explicit loops over data points and neurons, primarily for educational clarity.
- **MiniSom library:** Minimalistic NumPy-based package providing efficient vectorized updates, supporting both rectangular and hexagonal topologies, with control over key hyperparameters.

For MiniSom, an 8×8 grid was used for the Iris dataset and a 10×10 grid for the blob dataset, balancing resolution and computational cost. Both implementations use random initialization from a uniform distribution over $[0, 1]^d$, although MiniSom also supports PCA-based initialization for faster convergence.

3.3 Hyperparameters and Evaluation Metrics

Table 1 shows the experimental hyperparameters. The initial learning rate α_0 was set to 0.5 and the initial radius σ_0 proportional to the maximum grid dimension, following common practice.

Table 1: SOM hyperparameters for experiments.

Parameter	**Iris dataset**	**Blob dataset**
Grid size	8×8	10×10
Initial learning rate α_0	0.5	0.5
Initial radius σ_0	4.0	5.0
Iterations	5000	5000
Topology	Rectangular	Hexagonal
Initialization	Random	Random

Two quantitative measures were used:

- **Quantization error:** Average distance between each input and its BMU's weight vector:

$$E_{\text{quant}} = \frac{1}{N} \sum_{n=1}^{N} \|x_n - w_{c(n)}\|_2,$$

  where $c(n)$ is the BMU for sample x_n.
- **Cluster purity:** For labeled Iris data, assign each neuron the majority class label of mapped samples and count correctly labeled samples.

4 Results and Analysis

4.1 Visualization-Based Analysis

Figures 1 and 2 show U-matrices [3] (distance maps) produced by MiniSom. Darker regions indicate large distances between neighboring weight vectors (cluster boundaries), while lighter regions show homogeneous clusters.

For Iris, three prominent basins separated by darker ridges appear, roughly matching the three species. Interestingly, Setosa forms a clearly isolated cluster while the other two overlap more.

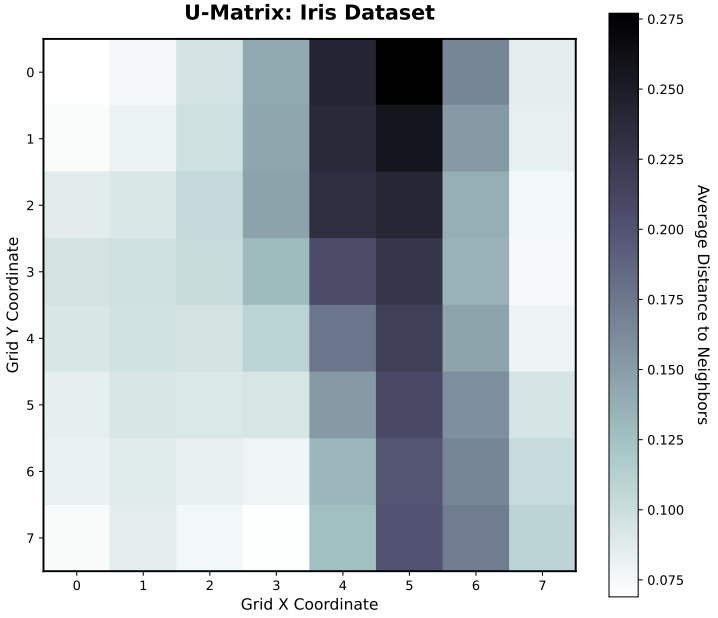


Figure 1: U-matrix for Iris dataset trained with an 8×8 SOM.

Hit maps show sample counts per neuron, revealing that most neurons are used but with varying densities. For the blob dataset, high hit counts concentrate in four well-separated regions matching the underlying clusters, while intermediate regions have few hits due to absent data. Component planes for Iris features show smooth gradients, indicating that the SOM learned an ordered representation where neighboring neurons respond to similar feature combinations.

4.2 Quantitative Evaluation

Table 2 reports quantization errors for both implementations across grid sizes on the Iris dataset. The MiniSom implementation consistently achieves slightly lower error, likely due to vectorized updates and careful parameter handling, although the differences are modest.

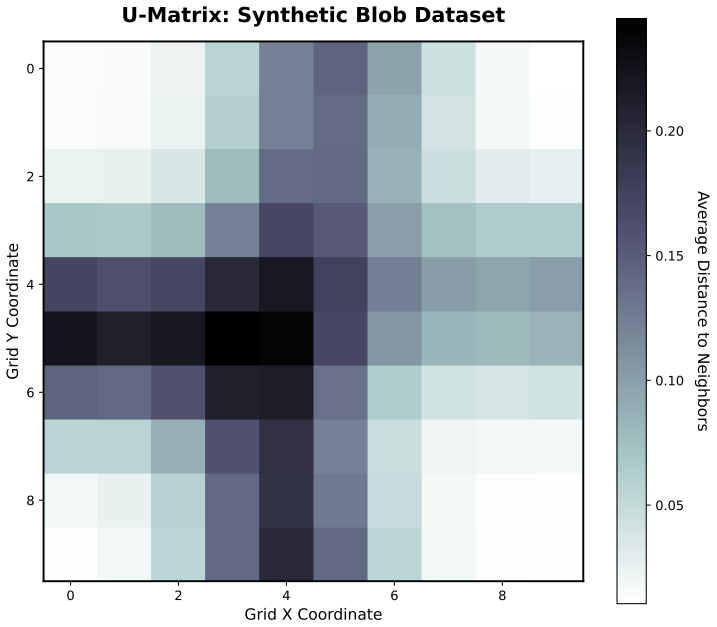


Figure 2: U-matrix for synthetic blob dataset trained with a 10×10 SOM.

Table 2: Quantization error for different grid sizes (Iris dataset).

Grid size	**From-scratch**	**MiniSom**
6×6	0.78	0.74
8×8	0.65	0.61
10×10	0.62	0.59

Cluster purity on Iris reaches 98% for the 8×8 grid (147/150 correct), with only three misclassifications. The blob dataset achieved 100% purity (300/300 correct).

4.3 Effect of Hyperparameters

4.3.1 Learning Rate

Varying the initial learning rate α_0 shows that small values (e.g., 0.1) cause slow convergence and higher final error, while large values (e.g., 0.9) produce oscillatory behavior and poorly ordered maps. In these experiments, values around 0.3–0.5 with exponential decay provided a good stability–convergence tradeoff.

4.3.2 Neighborhood Radius

The initial neighborhood radius σ_0 strongly affects global organization quality. Too small a σ_0 fragments the map into locally ordered but globally disordered clusters, while too large a value delays sharp cluster boundary formation. A heuristic choice of σ_0 equal to half the maximum grid dimension, with exponential decay, produced well-ordered maps for both datasets.

5 Conclusion

This report presented an overview of Self-Organizing Maps, including their architecture, training algorithm, and relation to other unsupervised methods, together with practical Python implementations and benchmark experiments. Visualizations (U-matrices, hit maps, component planes) showed that SOMs effectively reveal cluster structure and feature relationships. Quantitative metrics (quantization error, cluster purity) confirmed that reasonably small grids model the datasets well. The comparison between the from-scratch implementation and the MiniSom library indicated that higher-level libraries can offer modest performance gains and convenient interfaces without altering underlying learning behavior.

6 References

References

[1] Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1), 59–69.

[2] Kohonen, T. (2001). *Self-Organizing Maps* (3rd ed.). Springer.

[3] Vesanto, J., & Alhoniemi, E. (2000). Clustering of the self-organizing map. *IEEE Transactions on Neural Networks*, 11(3), 586–600.

[4] Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.